Estudio completo del motor del liquidador de retenciones 2026

Documento de estudio del script `motor_liquidador.py`:
librerías, reglas tributarias, arquitectura, tutorial por línea de comandos y casos de prueba

Proyecto: Liquidador de retenciones Colombia 2026 — repositorio `liquidador-retenciones`

Corte normativo: 23 de octubre de 2026 | UVT 2026: $52.374 | Versión del script: 2026.1
Documento generado el 28 de octubre de 2026
Material educativo: no constituye asesoría tributaria

Índice

1. Propósito y alcance del estudio
2. Qué es motor_liquidador.py y su relación con el diseño funcional y el notebook
3. Estructura del script
4. Por qué cada librería y módulo de Python
5. Arquitectura: el motor y la línea de comandos
6. Las reglas tributarias: por qué existen y cómo están implementadas
7. Tutorial de uso paso a paso
8. Contrato de entradas y salidas para un escenario real
9. Los 12 casos de prueba y su lectura
10. Criterios de aceptación
11. Limitaciones y camino a producción
12. Glosario
13. Referencias del proyecto y normativas

1. Propósito y alcance del estudio

Este documento estudia el script `motor_liquidador.py`: la implementación ejecutable del diseño funcional del liquidador de retenciones (`docs/diseno-funcional.md`) lista para usarse fuera de Jupyter, desde la línea de comandos, con entrada en JSON y salidas en JSON y CSV. El objetivo del estudio es triple:

1. **Explicar el porqué de cada pieza técnica**: qué librerías se usan, por qué se eligieron y qué problema resuelve cada una (nada está "por costumbre").
2. **Explicar el porqué de cada regla tributaria**: qué dice la norma, por qué la regla se implementó de esa forma y cómo se verifica en el código.
3. **Enseñar a usarlo**: tutorial de línea de comandos, formato de entrada, archivos de salida, códigos de salida y lectura de los 12 casos de prueba y de los criterios de aceptación.

Todo lo que aquí se describe corresponde exactamente al código del script. La verificación incluida en el propio archivo (`motor_liquidador.py casos`) da ✅ para los 12 casos del diseño funcional y sale con código 0.

> **Advertencia.** Este es un proyecto educativo. No reemplaza al contador ni la tabla oficial de retenciones de la DIAN. Antes de usar una regla en una empresa real, verifique el RUT, el concepto, el municipio, la fecha de la operación y la norma vigente. Los fundamentos legales citados provienen de la documentación del proyecto y deben confrontarse con la norma actualizada.

2. Qué es motor_liquidador.py y su relación con el diseño y el notebook

El diseño funcional define el liquidador como una **tabla de decisión con memoria**: un sistema que debe saber qué regla aplica, con qué base, con qué tarifa, desde cuándo, con qué documento y con qué asiento. Y agrega una regla de oro: *si no puede explicar una regla, no debe automatizarla*.

El script materializa esa idea en un único archivo Python sin dependencias externas. Es exactamente el mismo motor que quedó documentado en `liquidador.ipynb` (mismas clases, mismas tablas, mismos mensajes), pero empaquetado para un caso real:

- **Entrada:** archivo JSON con comprador, proveedor y una o varias operaciones (lotes).
- **Salida:** resumen por consola y archivos JSON (liquidación con auditoría) y CSV (asientos y certificados) listos para Excel o el ERP.
- **Durabilidad:** repositorio de facturas liquidadas persistido en archivo, para evitar duplicados entre corridas.
- **Actualización normativa:** UVT, retefuente e ICA pueden reemplazarse con un JSON sin tocar el código.
- **Automatización:** códigos de salida 0 / 1 / 2 para integrarlo en procesos por lotes o cron.

Concepto del diseño	Sección del diseño	Dónde vive en el script
Entradas del sistema (comprador, proveedor, operación, reglas)	§3	Dataclasses `Comprador`, `Proveedor`, `Operacion`, tablas normativas y conversores `*_desde_dict`
Salidas (liquidación, asientos, documentos, alertas, trazabilidad)	§4	Objeto `Liquidacion`, `Asiento`, `CertificadoRetencion`, `ItemChecklist`, `RegistroAuditoria`; archivos JSON/CSV con `guardar_liquidacion`
Tabla de decisión (14 filas)	§5	`TABLA_DECISION` y `buscar_fila_decision`
Reglas de negocio R1–R20	§6	`validar_entradas`, `normalizar_factura` y `LiquidadorRetenciones.calcular`
Validaciones y mensajes	§7	Objeto `Alerta` (Info / Alerta / Bloqueo)
Modelo de datos conceptual	§8	Las mismas dataclasses, que son el modelo ya tipado
Pantalla de "explicación"	§9	Función `explicar(liq)`, impresa por consola
12 casos de prueba	§10	`casos_diseno()` + comando `casos`
10 criterios de aceptación	§11	Verificados por `casos` y por las pruebas del proyecto
Plantilla de regla y trazabilidad	§14, §4.5	`ReglaRetefuente` / `ReglaICA` y `RegistroAuditoria`

> **Relación con el notebook.** El notebook queda como documento didáctico con resultados guardados; el script es la misma lógica lista para integrar. La única diferencia funcional es la capa de entrada/salida: el notebook llama a `calcular()` con objetos Python; el script los construye desde JSON y persiste el resultado.

3. Estructura del script

El archivo se lee de arriba hacia abajo y tiene tres capas: **núcleo tributario** (lo que liquida), **presentación** (texto para humanos) y **línea de comandos** (JSON de entrada, archivos de salida y persistencia).

Bloque	Contenido	Para qué sirve
Docstring	Qué es, ejemplos de uso y códigos de salida	Documentación del propio archivo (`--help` la repite).
Imports y utilidades	`Decimal` (`D`, `dinero`, `redondear`), `cop`, `porcentaje`, `normalizar`	Aritmética monetaria exacta, formatos y comparación sin tildes.
Constantes	`UVT_2026`, `VERSION_TABLAS`, `TABLA_UVT`, tarifas de IVA y reteIVA	Los parámetros normativos que cambian con el tiempo.
Enums y excepciones	`TipoPersona`, `Regimen`, `TipoDocumento`, `Concepto`, `Severidad`, `Alerta`, `CalculoBloqueado`, `RevisionHumana`, `ReglaNoEncontrada`	Catálogos cerrados y la forma de decir "no puedo explicar esto".
Modelo de datos	`Comprador`, `Proveedor`, `Operacion`, `DatosFactura`, `ReglaRetefuente`, `ReglaICA`, `FilaDecision`, `Asiento`, `LineaAsiento`, `CertificadoRetencion`, `ItemChecklist`, `RegistroAuditoria`, `Liquidacion`	Las entidades del diseño funcional ya tipadas.
Tablas normativas	`TABLA_RETEFUENTE` (con vigencias), `TABLA_ICA`, `buscar_regla_retefuente`, `buscar_regla_ica`, `categoria_reteiva`, `categoria_ica`, `minimo_reteiva_uvt`	De dónde sale cada tarifa, mínimo y fundamento.
Tabla de decisión	`TABLA_DECISION` (14 filas), `buscar_fila_decision`	El escenario pedagógico que explica cada liquidación.
Cuentas contables	`CUENTAS`, `cuenta_gasto`, `cuenta_retefuente`	Mapa concepto → cuenta de gasto/inventario y cuenta de pasivo.
Normalización	`normalizar_factura`	Base, IVA y total con o sin IVA incluido, descuentos y TRM.
Validaciones	`validar_entradas`, `_exigir_sin_bloqueos`	NIT, fechas, tarifas, banderas y casos fuera de alcance.
Motor	`RepositorioFacturas`, `LiquidadorRetenciones.calcular`	Retefuente, reteIVA, reteICA, neto, asientos, certificados, checklist y auditoría.
Presentación	`explicar`, `borradores_declaracion`	Texto legible de la liquidación y resumen para el Formulario 350 / declaración municipal.
Datos de prueba	`CasoPrueba`, fábricas de compradores/proveedores, `operacion()`, `casos_diseno()`	Los 12 casos del diseño con sus valores esperados.
Serialización	`_json_default`, `liquidacion_a_dict`, `guardar_json`	JSON legible y escritura atómica (temporal + renombrado).
Persistencia	`RepositorioFacturasArchivo`	Registro duradero de facturas liquidadas (evita duplicados entre corridas).
Conversores	`comprador_desde_dict`, `proveedor_desde_dict`, `operacion_desde_dict`, `cargar_entrada`, `cargar_tablas`	Del JSON de entrada al modelo, con mensajes claros por campo.
Salida a archivos	`imprimir_*`, `guardar_liquidacion`	Consola legible y CSV de asientos y certificados

		(separador ;, UTF-8 con BOM).
Comandos	`plantilla`, `plantilla-tablas`, `casos`, `tablas`, `calcular`, `construir_parser`, `main`	La interfaz de línea de comandos con argparse y códigos de salida.

4. Por qué cada librería y módulo de Python

El script usa **solo la biblioteca estándar**. Esto es intencional: un liquidador tributario debe ser portable, auditable y sin dependencias que cambien el resultado. No requiere instalación: Python 3.10 o superior y listo.

4.1 decimal — la aritmética del dinero

El dinero **nunca** se calcula con `float`. Los números de punto flotante son binarios y no representan exactamente los decimales; por ejemplo:

```
0.1 + 0.2          # en float → 0.30000000000000004
Decimal("0.1") + Decimal("0.2")   # → Decimal('0.3'), exacto
```

En impuestos, ese error de centavos se acumula y rompe el cuadre del asiento. Por eso:

- `Decimal` guarda los valores como texto decimal y suma, multiplica y divide sin redondeos ocultos.
- La función `D(x)` convierte con `Decimal(str(x))`: nunca se pasa por `float`, ni siquiera si el JSON trae `0.19`.
- `dinero()` / `redondear()` aplican la **política única de redondeo** exigida por el diseño: dos decimales con `ROUND_HALF_UP` (redondeo comercial: 0,005 sube).

El caso 4 demuestra el resultado: $104.748 × 4% = $4.189,92 exacto. Toda comparación en las pruebas usa `Decimal`.

4.2 dataclasses — el modelo de datos tipado

`@dataclass` genera el constructor, la representación y la comparación de las entidades del diseño (§8: Comprador, Proveedor, Factura/Operación, Regla, Asiento…). Dos decisiones importantes:

- Las entradas (`Comprador`, `Proveedor`, `Operacion`) son `frozen=True`: inmutables. El motor no puede "por accidente" cambiar el NIT o la base mientras liquida. Esto es clave para la auditoría.
- Las salidas usan `field(default_factory=list)` para listas (asientos, alertas, certificados…), evitando el error clásico de compartir una lista mutable entre instancias.

`asdict()` convierte la liquidación completa en diccionarios para el archivo JSON de salida.

4.3 enum — catálogos cerrados

Las listas del diseño se modelan con `Enum`: `TipoPersona`, `Regimen`, `TipoDocumento`, `Concepto` y `Severidad`. Ventajas:

- No se puede escribir "servicio general" con un error de tipeo: el conversor lo rechaza antes de calcular y lista las opciones válidas.
- Las búsquedas en las tablas comparan enums, no texto libre: imposible que "Servicios" y "servicio" generen dos reglas distintas.
- Heredan de `str`: se imprimen con su nombre en español.

4.4 datetime — vigencias y sello de auditoría

- La regla R16 exige **vigencia desde/hasta**: cada `ReglaRetefuente` guarda `vigencia_desde` y `vigencia_hasta` y `vigente(fecha)` responde si la regla aplica a la fecha de emisión. Con esto el motor no puede aplicar el mínimo de 4 UVT cuando ya rige el de 2 UVT (o al revés).
- La regla R20 exige **quién y cuándo**: la auditoría guarda `datetime.now()` y el repositorio de duplicados registra la fecha de cada factura liquidada.
- Las fechas de emisión y pago se comparan para validar coherencia.

4.5 json — entrada y trazabilidad serializable

El JSON cumple dos papeles: es el formato de entrada de las facturas y el formato de salida de la auditoría.

- `json.loads` lee la entrada con validación estricta (un JSON malo produce error claro y código de salida 1).
- `json.dumps(..., default=_json_default, ensure_ascii=False, indent=2)` serializa `Decimal`, fechas y enums como texto; `ensure_ascii=False` conserva tildes y "ñ" legibles (razón social del proveedor).
- La auditoría guarda `entrada_json` y `resultado_json`: el mismo JSON de salida es la trazabilidad.

4.6 hashlib — integridad de la entrada

El motor calcula `sha256` sobre el JSON de entrada y lo guarda como `hash_entrada`. Si alguien cambia después un dato (base, fecha, bandera), el hash cambia y la manipulación queda en evidencia. Es la "memoria" de la tabla de decisión: cada resultado queda atado a los datos exactos que lo produjeron.

4.7 unicodedata — comparar textos sin tildes

`normalizar()` usa `NFKD` y elimina los caracteres de combinación para que "Bogotá" y "bogota" (o "Medellín" sin tilde) se comparen igual. Se usa en la búsqueda de la tarifa de ICA, en el repositorio de duplicados y en los alias de los enums. Sin esto, "Bogotá" no encontraría la tarifa municipal por una tilde.

4.8 typing y future annotations — el contrato legible

Las firmas declaran tipos (`def calcular(...) -> Liquidacion`, `Sequence[ReglaRetefuente]`...). Con `from __future__ import annotations` se pueden usar anotaciones modernas sin costo en tiempo de ejecución. El tipado es el contrato entre el diseño y el código.

4.9 argparse — una interfaz de comandos profesional

`argparse` construye los subcomandos (`plantilla`, `plantilla-tablas`, `casos`, `tablas`, `calcular`), valida los argumentos, muestra la ayuda y la versión. El parser se subclasificó (`_Parser.error`) para que los errores de uso salgan con código 1 y no se confundan con los bloqueos tributarios (código 2).

4.10 logging — mensajes separados de los resultados

Los mensajes operativos (archivos leídos, tabla cargada, resumen) van por `logging` a `stderr`; los resultados tributarios van a `stdout`. Así, en un proceso por lotes se puede capturar solo la salida útil (`2>/dev/null`) o solo los errores, y `--verboso` activa el nivel DEBUG para diagnosticar.

4.11 csv y pathlib — salidas listas para Excel y ERP

Los asientos y certificados se exportan con el módulo `csv` usando separador `;` y codificación `utf-8-sig` (con BOM): se abren directo en Excel en español sin caracteres rotos y quedan listos para importar al sistema contable. `pathlib.Path` maneja las rutas de forma portable, crea el directorio de salida y permite la escritura atómica (escribir un temporal y renombrarlo), de modo que un corte de energía nunca deja un JSON a medio escribir.

4.12 sys — el código de salida es parte del contrato

`sys.exit(main())` convierte el resultado en un código para el sistema operativo: `0` todo liquidado, `1` error de entrada o archivos, `2` operaciones bloqueadas por el motor. Un cron o un CI puede decidir automáticamente si continuar o alertar.

4.13 Lo que se dejó afuera (y por qué)

- **pandas e IPython:** en el notebook solo servían para dibujar tablas. El script imprime en texto plano, así que se eliminaron: cero dependencias externas.
- **float:** por el problema binario ya explicado.
- **numpy:** no hay álgebra ni vectores; Decimal es la herramienta correcta para dinero.

- **Frameworks web o base de datos:** la v1 es un motor determinista por archivos; la persistencia se resuelve con JSON/CSV y un repositorio de duplicados en archivo (sección 11 explica el salto a producción).

5. Arquitectura: el motor y la línea de comandos

Hay dos recorridos: el del **motor** (qué pasa con una factura) y el de la **CLI** (qué pasa con un archivo de entrada).

5.1 El recorrido de una liquidación (motor)

1. Validar entradas → 2. Revisar duplicados → 3. Normalizar factura → 4. UVT y pago → 5. Retefuente → 6. ReteIVA → 7. ReteICA → 8. Neto → 9. Asientos, certificados, checklist, auditoría

1. **Validación.** `validar_entradas()` revisa NIT, nombres, fechas, tarifa de IVA, banderas y casos fuera de alcance. Si hay un `Bloqueo`, `_exigir_sin_bloqueos()` lanza `CalculoBloqueado` con todas las alertas. Es la regla de alcance del diseño: "mejor no calcular que calcular mal".
2. **Duplicados.** Si hay un repositorio y ya existe proveedor + número, bloquea (salvo `autorizar_duplicado=True`).
3. **Normalización.** `normalizar_factura()` deja una sola verdad: base bruta, descuentos, base, IVA y total. Acepta base directa, total + IVA, o total con IVA incluido (base = total / (1 + tarifa)); convierte USD con TRM.
4. **UVT y pago.** Toma la UVT del año de emisión; si no existe, bloquea. Calcula el factor de pago: 1 si no se informa valor pagado, o `valor_pagado / total` si hay pago parcial. Las retenciones se calculan sobre la **parte pagada**.
5. **Retefuente.** Si el comprador no es agente, o el proveedor es SIMPLE o autorretenedor, es $0 con alerta. Si no, busca la regla vigente por concepto y condición; compara la base con el mínimo (si es inferior, $0); si cumple, aplica la tarifa.
6. **ReteIVA.** Exige comprador agente + proveedor responsable + IVA > 0 + base por encima del mínimo (2 UVT servicios / 10 UVT compras). Se calcula 15% del IVA pagado.
7. **ReteICA.** Exige comprador agente + actividad gravada + proveedor no SIMPLE; busca la tarifa por municipio y actividad (si no existe, bloquea: "no se inventa la tarifa"); aplica la tarifa a la base pagada.
8. **Neto.** `neto = total_pagado − retefuente − reteIVA − reteICA`; si es negativo, bloquea.
9. **Evidencia.** Construye asientos (causación, pago, pago de retenciones), certificados (solo si hay retención), checklist y auditoría. Si un asiento no cuadra, lanza excepción. Al final registra la factura en el repositorio.

5.2 El recorrido de la línea de comandos (CLI)

argparse → leer JSON → conversores → objetos → tablas y repositorio → motor (por operación) → consola + JSON/CSV → código de salida

1. **argparse** valida los argumentos y elige el subcomando.
2. **cargar_entrada** lee y valida la estructura del JSON.
3. **comprador_desde_dict / proveedor_desde_dict / operacion_desde_dict** convierten el JSON al modelo, con mensajes que nombran el campo exacto que falta o está mal.
4. **cargar_tablas** (si se pidió `--tablas`) reemplaza UVT, retefuente e ICA; **RepositorioFacturasArchivo** (si se pidió `--repositorio`) carga el registro previo de duplicados.
5. Para cada operación se llama al motor. Si bloquea, se imprime el motivo con sus alertas y se continúa con la siguiente (un lote no se cae por una factura mala).
6. Cada liquidación exitosa se imprime y, salvo `--solo-mostrar`, se guarda en JSON y CSV.
7. Al final se imprime el resumen y se sale con 0 (todas bien) o 2 (hubo bloqueos). Errores de entrada o archivos salen con 1.

6. Las reglas tributarias: por qué existen y cómo están implementadas

Esta sección recorre las reglas R1–R20 del diseño funcional (§6) y las conecta con la norma, con el porqué de la regla y con su implementación exacta en el script.

6.1 UVT y mínimos (contexto de todas las reglas)

La UVT es la unidad de valor tributario: los mínimos de retención se expresan en UVT para que la norma no quede obsoleta con la inflación. El script carga la UVT por año en `TABLA_UVT = {2026: 52.374}` y la busca por la fecha de emisión. Derivados: 2 UVT = $104.748 y 10 UVT = $523.740. Con `--tablas` se puede cambiar la UVT o agregar años sin tocar el código.

El mínimo de servicios tiene historia, y por eso está modelado como vigencias:

Regla	Vigencia	Mínimo servicios	Por qué
RF-SERV-001/002	01/01/2026 – 07/05/2026	2 UVT	Regía el Decreto 572 de 2025.
RF-SERV-003/004	08/05/2026 – 30/06/2026	4 UVT	El Consejo de Estado suspendió los arts. 2–8 del Decreto 572 (mínimo temporal de 4 UVT).
RF-SERV-005/006	01/07/2026 – 31/12/2026	2 UVT	El Consejo de Estado revocó la suspensión con efectos desde el 1/07/2026: vuelve 2 UVT.

Por qué se implementa así: la regla R16 obliga a que cada regla tenga fecha desde/hasta, y el diseño lo subraya con un ejemplo: "si el liquidador no guarda la fecha de vigencia, puede aplicar una base de 4 UVT cuando ya rige la de 2 UVT". La búsqueda `buscar_regla_retefuente` filtra por concepto, condición del proveedor y fecha, y si no encuentra nada, lanza `ReglaNoEncontrada`.

6.2 IVA (regla R8)

- **Norma y porqué:** el IVA se causa sobre la base gravable; la base de las retenciones es el valor *antes* de IVA. La tarifa general es 19%; el modelo también acepta 5% y 0% (excluido/exento).
- **Implementación:** `TARIFAS_IVA_PERMITIDAS = {0.19, 0.05, 0}`. Si la factura trae base: IVA = base × tarifa y se valida el total. Si el total incluye IVA: base = total / (1 + tarifa) e IVA = total − base, y se validan base e IVA informados contra los derivados (tolerancia de 1 centavo).
- **Error clásico que evita:** calcular la retención sobre el total con IVA. El motor siempre retiene sobre la base sin IVA.

6.3 Retefuente de renta (reglas R1, R2, R3, R4)

R1 — separar las tres calidades. Agente de retefuente, de reteIVA y de reteICA son preguntas independientes en el modelo (`agente_retefuente`, `agente_reteiva`, `agente_reteica`). Por qué: una SAS normal retiene renta pero no reteIVA; un gran contribuyente retiene ambos; un agente municipal solo reteICA. Inferir una calidad de otra es el error más caro del proceso manual.

R2 — base. La base de retefuente es el valor antes de IVA menos descuentos y devoluciones: `datos.base` (ya descontados los descuentos) y `base_pago` si hay pago parcial. El IVA nunca entra en la base.

R3 — tarifa. Se toma de `TABLA_RETEFUENTE` por concepto y condición (declarante / no declarante). Sin regla: bloqueo.

Concepto	Declarante	No declarante	Mínimo
Servicios en general	4%	6%	2 UVT ($104.748), con la línea de tiempo anterior
Honorarios	11%	10%	Sin mínimo (0 UVT)
Arrendamiento de vivienda	3,5%	3,5%	10 UVT ($523.740); IVA excluido

Arrendamiento de bienes muebles	4%	4%	Sin mínimo
Compra de bienes	2,5%	3,5%	10 UVT ($523.740)
Transporte de carga	1%	1%	2 UVT
Hoteles y restaurantes	3,5%	3,5%	2 UVT

R4 — mínimo. Antes de calcular se compara la base pagada con el mínimo (`cumple_minimo_rf = base_pago >= minimo_rf_valor`). Si es inferior: retención $0 con alerta. La comparación es "inferior a" → si la base es *igual* al mínimo, sí se retiene: ese es exactamente el caso 4 del diseño ($104.748 retiene $4.189,92).

> **Detalle de honorarios.** El fundamento de las reglas RF-HON cita el Art. 383 ET, que prevé una excepción para honorarios (3.300 UVT). La v1 **no implementa esa excepción**: la deja documentada en el fundamento para que el contador la aplique cuando corresponda. Es uno de los puntos del "camino a producción" (sección 11).

6.4 Banderas del proveedor (reglas R5, R6, R7)

Bandera	Efecto en el motor	Por qué
SIMPLE (R5)	Retefuente = 0 y reteICA = 0 con alerta. ReteIVA solo si es responsable de IVA y el comprador es agente.	Art. 911 ET: al contribuyente del régimen SIMPLE no se le practica retención a título de renta ni de ICA; la reteIVA sí procede en ciertos casos (compra a responsable de IVA del SIMPLE por un agente).
Autorretenedor (R6)	Retefuente = 0 con alerta; reteIVA y reteICA pueden aplicar. Si solo tiene autorretención especial (código 59) y no la general (código 15), el motor *bloquea*: no asume la exención.	El autorretenedor general se retiene a sí mismo: retenerle sería doble retención. La especial no cubre todos los pagos, así que asumir exención sería un error; por eso se deriva a revisión humana.
No residente (R7)	Bloquea el cálculo: el caso se deriva al módulo de pagos al exterior.	Las tarifas nacionales no aplican: hay reglas de pagos al exterior, convenios y reteIVA del 100% que están fuera de la v1.
Declarante / no declarante	Elige la columna de tarifa (4% vs 6%; 2,5% vs 3,5%; 11% vs 10%).	La tarifa depende de la condición tributaria del beneficiario; además el motor exige soporte de no declarante y valida que una persona jurídica sea declarante.

6.5 ReteIVA (reglas R9 y R10)

R9. ReteIVA = 15% del IVA, solo si: comprador agente de reteIVA, proveedor responsable de IVA, operación gravada (IVA > 0), base por encima del mínimo (2 UVT servicios / 10 UVT compras) y sin exclusión. El código pide esas condiciones en cadena y emite una alerta distinta para la que falle (`RIVA-AGENTE`, `RIVA-RESPONSABLE`, `RIVA-SIN-IVA`, `RIVA-MINIMO`).

- **Por qué "comprador agente" no se infiere:** el error clásico es creer que todo responsable de IVA es agente de reteIVA. No lo es: una SAS normal no practica reteIVA en una compra ordinaria. Por eso la calidad es una bandera explícita.
- **Compra a SIMPLE:** el comprador agente sí practica reteIVA cuando el proveedor SIMPLE es responsable de IVA (caso 8: reteIVA $28.500). El código lo cubre porque la única condición del proveedor para reteIVA es `responsable_iva`.
- **Exclusión entre grandes contribuyentes:** el diseño excluye de la v1 las operaciones entre grandes contribuyentes con reglas especiales. El motor bloquea con `RETEIVA-GC` cuando comprador y proveedor son ambos grandes contribuyentes. Los campos `entidad_estatal` y `emisor_tarjetas` existen en el modelo para completar esa exclusión en una versión posterior.

R10. La reteIVA especial del 100% (servicios de no residentes, entre otros) está fuera de la v1: el proveedor no residente ya bloquea en R7.

6.6 ReteICA (regla R11)

ReteICA = base × tarifa municipal, solo si: el comprador es agente en ese municipio, la actividad del proveedor está gravada, la base supera el mínimo municipal y el proveedor no es SIMPLE.

- La tarifa se busca por **municipio del comprador + actividad del proveedor** (`categoria_ica(concepto)` → "Servicios" o "Compras"), normalizando tildes.
- Si no existe la regla, el motor **bloquea** con el mensaje "No se inventa la tarifa". La tabla de la v1 solo trae la tarifa pedagógica de Bogotá para servicios (0,966%, Acuerdo 724 de 2026, mínimo no configurado). Con `--tablas` se cargan las tarifas reales de cada municipio.
- La reteICA no va en el Formulario 350: va en la declaración municipal. El motor la separa en sus asientos (cuenta 2368) y en los borradores.

6.7 Neto y redondeo (reglas R12 y R13)

- **R12:** neto = total pagado − suma de las tres retenciones; si da negativo, bloqueo (una retención no puede superar el pago).
- **R13 y política de redondeo:** cada valor monetario se cuantiza a 2 decimales con `ROUND_HALF_UP` en un único lugar (`dinero()`). El caso 4 muestra valores exactos ($4.189,92 / $2.985,32) y el diseño pide registrar la política; el script la documenta como constante `CENTAVO` y la aplica siempre igual en el cálculo, el asiento y el certificado.

6.8 Asientos y cuentas (regla R13)

El motor genera tres asientos (causación, pago, pago de retenciones) y exige `débitos = créditos`; si no cuadran, lanza `CalculoBloqueado`. La regla de oro contable del proyecto: **la retención no es un gasto, es un pasivo** mientras se declara y paga. Por eso la retención nace al crédito de una cuenta 236x y se extingue con débito en el asiento de pago de retenciones.

Cuenta	Uso en el script
5135 Servicios / 5110 Honorarios / 5120 Arrendamientos / 143505 Inventarios	Débito de la causación según el concepto.
240805 IVA descontable	Débito de la causación si el comprador es responsable de IVA y hay IVA; si no, el IVA va al costo/gasto.
220505 Proveedores nacionales	Crédito en causación; débito en el pago.
111005 Bancos	Crédito del pago (neto) y del pago de retenciones.
236525 / 236515 / 236530 / 236540	Pasivo de retefuente según concepto (servicios, honorarios, arrendamientos, compras).
236701	Pasivo de reteIVA (Impuesto a las ventas retenido).
2368	Pasivo de reteICA (ICA retenido).

Cada línea lleva cuenta, nombre, débito/crédito, tercero (proveedor), centro de costo (`centro_costo` de la operación, por defecto "ADMIN"), descripción y documento, tal como exige el diseño en su sección 4.2. Esas mismas líneas son las que se exportan al CSV de asientos.

6.9 Certificados (regla R14)

Solo se emiten si la retención es mayor que cero: renta si retefuente > 0, reteIVA si reteIVA > 0, reteICA si reteICA > 0. Cada `CertificadoRetencion` guarda agente, retenido, concepto, base, tarifa, valor, fundamento, ciudad y fecha, y se **valida antes de entregarse** (si falta un campo, el motor lanza excepción).

- El certificado de renta expone `requisitos_art_381()`, los siete datos del Art. 381 ET: nombre y NIT del agente, nombre y NIT del retenido, concepto, base, tarifa, valor y año gravable.
- El certificado de reteIVA discrimina base sin IVA, IVA, tarifa (15%) y valor, como exige el diseño (y el Decreto 1625 de 2016, art. 1.6.1.12.13, según la documentación del proyecto).

6.10 Documentos y checklist (regla R15)

Antes de pagar, el sistema verifica: RUT del proveedor, factura validada, orden de compra, acta de recibido, cálculo de retención y asiento. El `checklist` marca cada documento como obligatorio y recibido, y `puede_pagar` responde False si falta alguno (por ejemplo, sin RUT) o si hay alertas de bloqueo. Si no hay retención no aplica su certificado, y el borrador del Formulario 350 o de la declaración municipal solo aparece cuando hay qué declarar.

6.11 Vigencia (regla R16)

Toda regla tiene desde/hasta y toda liquidación guarda la UVT usada y la versión de tablas (`"2026.1"`). Una operación de 2027, por ejemplo, bloquea porque no hay UVT cargada para ese año. Cambiar la UVT o la norma es cambiar la tabla (por `--tablas`), no el código.

6.12 Validaciones de datos (regla R17)

Validación	Comportamiento
NIT y razón social no vacíos (comprador y proveedor)	Bloqueo
Número de documento de la operación	Bloqueo si falta (también alimenta duplicados)
Fecha de pago ≥ fecha de emisión; sin cruce de año	Bloqueo (el cruce de año exige revisar UVT y vigencia)
Tarifa de IVA permitida (19%, 5%, 0%)	Bloqueo
Base > 0; descuentos coherentes; total = base + IVA	Bloqueo
Moneda COP o USD con TRM > 0	Bloqueo si USD sin TRM; alerta de conversión simple
Proveedor identificado y residente	Bloqueo
RUT recibido / factura validada / periodo abierto / retención no practicada	RUT: alerta y `puede_pagar=False`; los demás: bloqueo

6.13 Duplicados y periodo cerrado (reglas R18 y R19)

El repositorio guarda la pareja (proveedor, número) y bloquea el segundo cálculo sin autorización explícita. En la CLI, `RepositorioFacturasArchivo` lo persiste en JSON entre corridas (registra usuario y fecha), y `--autorizar-duplicado` permite el recálculo consciente. El campo `periodo_cerrado` bloquea cambios sin ajuste autorizado. Ambas reglas protegen contra el doble registro de la misma retención, que es uno de los errores más difíciles de corregir después de declarar.

6.14 Auditoría (regla R20)

El `RegistroAuditoria` guarda: usuario, fecha y hora, versión de tablas, UVT y año, regla aplicada, fundamento legal, hash de la entrada, entrada JSON, resultado JSON, documentos generados y el cambio realizado. En la CLI, el usuario llega por `--usuario` y todo el objeto queda dentro del archivo `liquidacion_*.json` de salida. Es la memoria de la tabla de decisión: permite reconstruir por qué se pagó lo que se pagó. Antes de liquidar un lote, el comando `tablas` muestra qué tablas están activas (secciones 6.16 y 7.8).

6.15 La tabla de decisión (14 filas)

La tabla de la sección 5 del diseño se conserva como dato (`TABLA_DECISION`) y la función `buscar_fila_decision` elige la fila que describe la combinación. Es pedagógica: si la combinación no está tabulada, devuelve `None` y el motor calcula igual con las reglas, dejando una alerta `DECISION-NO-TABULADA`. El número de fila aparece como "Escenario" en la salida.

#	Comprador	Proveedor	Concepto	RF	ReteIVA	ReteICA

1	Agente RF, no reteIVA, no reteICA	Declarante	Servicio	4%	No	No
2	Agente RF, no reteIVA, no reteICA	No declarante	Servicio	6%	No	No
3	Agente RF, no reteIVA, no reteICA	Declarante	Honorarios	11%	No	No
4	Agente RF, no reteIVA, no reteICA	No declarante	Honorarios	10%	No	No
5	Agente RF, no reteIVA, no reteICA	Declarante	Arrendamiento vivienda	3,5%	No	No
6	Agente RF, no reteIVA, no reteICA	Declarante	Compra	2,5%	No	No
7	Agente RF, no reteIVA, no reteICA	No declarante	Compra	3,5%	No	No
8	Agente RF y reteIVA	Declarante	Servicio	4%	15%	No
9	Agente RF y reteIVA	Declarante	Compra	2,5%	15%	No
10	Agente RF y reteICA	Declarante	Servicio	4%	No	Tarifa municipal
11	Agente RF, reteIVA y reteICA	Declarante	Servicio	4%	15%	Tarifa municipal
12	Agente RF	SIMPLE	Servicio	0%	15% si es responsable de IVA	0%
13	Agente RF	Autorretenedor	Servicio	0%	15% si aplica	Según municipio
14	No agente RF	Cualquiera	Cualquiera	0%	Según agente	Según agente

6.16 Cómo validar las tablas contra la norma

Las tablas del motor son datos, no verdades: hay que confrontarlas con la norma vigente antes de operar. Cada tipo de dato tiene su fuente oficial:

Qué se valida	Fuente oficial
Tabla de retenciones (tarifas por concepto y condición, mínimos)	DIAN (dian.gov.co): la resolución anual del Ministerio de Hacienda–DIAN que fija la tabla vigente.
UVT del año (2026: $52.374)	DIAN: la resolución que fija la UVT de cada año.
Decretos, suspensiones y vigencias (Decreto 572 de 2025, Auto 30229 de 2026…)	SUIN-Juriscol (suin-juriscol.gov.co) y el Diario Oficial.
ReteIVA 15% / 100%, agentes y designaciones	Arts. 437-1 y 437-2 del ET y las resoluciones de designación de la DIAN.
Calidad del proveedor (SIMPLE, autorretenedor 15/59, gran contribuyente, declarante)	RUT del proveedor, consultado en la DIAN (MUISCA).
ReteICA por municipio (tarifa por actividad y mínimo)	El acuerdo municipal de cada alcaldía (ej. Acuerdo 724 de 2026 en Bogotá; en el ejemplo del video, el de Barranquilla).

- **Regla práctica:** cada fila cargada debe poder citar su fundamento y su vigencia contra una de esas fuentes; si no, no entra a producción.
- **Quién firma:** el revisor fiscal o el tributarista valida la tabla cargada, igual que el diseño exige firmar el alcance con un contador.
- **Cómo verificar lo que el motor tiene activo:** el comando `tablas` (o `tablas --tablas archivo.json`) imprime la versión, la UVT y cada regla con tarifa, mínimo, vigencia y fundamento.

Cada liquidación, además, guarda en su auditoría la versión de tablas y la UVT usadas: el JSON de salida es la evidencia de con qué reglas se calculó.

7. Tutorial de uso paso a paso

7.1 Requisitos

- Python 3.10 o superior. Sin dependencias externas: no hay que instalar nada.
- El archivo es ejecutable; si no, se corre con `python motor_liquidador.py …`.

7.2 Primeros pasos

```
./motor_liquidador.py --help          # ayuda general
./motor_liquidador.py plantilla      # escribe plantilla_factura.json de ejemplo
./motor_liquidador.py casos          # verifica los 12 casos del diseño (debe dar ✅ y salir con 0)
./motor_liquidador.py tablas         # muestra las tablas activas (UVT, tarifas, vigencias)
```

7.3 Liquidar una factura

1. Abra `plantilla_factura.json` y edite comprador, proveedor y la operación (valores ficticios en el ejemplo).
2. Ejecute:

```
./motor_liquidador.py calcular --entrada plantilla_factura.json \
    --usuario "contador@empresa.com" --salida salidas
```

La plantilla trae un servicio de $2.500.000 + IVA 19% de un proveedor declarante y un comprador SAS normal. El resultado esperado es: IVA $475.000, total $2.975.000, retefuente 4% = $100.000, neto $2.875.000, escenario 1, asientos cuadrados y "¿Autorizado para pagar? SÍ".

7.4 Formato del archivo de entrada (JSON)

El JSON tiene tres partes: `comprador`, `proveedor` y `operaciones` (lista; para una sola factura también se acepta la clave `operacion`). Fechas en formato `AAAA-MM-DD`; montos como número o texto sin separador de miles; booleanos como `true/false` o `"sí"/"no"`; enums por su nombre en español (acepta alias como "factura", "servicio", "compra", "honorario").

Campo	Defecto	Notas
comprador		
nit, razon_social	—	Obligatorios.
tipo	"Jurídica"	Jurídica / Natural.
agente_retefuente, agente_reteiva, agente_reteica	false	Las tres calidades se preguntan por separado (R1).
municipio	""	Define la tarifa de reteICA; obligatorio si es agente de reteICA.
responsable_iva, obligado_contabilidad	true	IVa descontable y alcance de la v1.
regimen	"Ordinario"	Ordinario / SIMPLE / Especial (SIMPLE bloquea: fuera de alcance).
direccion, gran_contribuyente, entidad_estatal, emisor_tarjetas	"" / false	Dirección para certificados; las banderas para reglas especiales.
proveedor		
nit, razon_social	—	Obligatorios.
tipo	"Jurídica"	Jurídica / Natural.

residente, declarante, responsable_iva	true	No residente bloquea; declarante elige la columna de tarifa.
simple, autorretenedor, autorretencion_especial, gran_contribuyente	false	Banderas que anulan o habilitan retenciones.
actividad_ciiu, municipio	""	Actividad para ICA y datos del certificado.
rut_recibido, soporte_no_declarante, actividad_gravada_ica	true	RUT faltante: alerta y sin autorización de pago; soporte faltante en no declarante: bloqueo.
operación		
tipo_documento	"Factura electrónica"	Factura / Nota crédito / Documento soporte (la nota crédito va por el ajuste, no por aquí).
numero	—	Obligatorio; alimenta el control de duplicados.
fecha_emision, fecha_pago	—	Obligatorias, AAAA-MM-DD; sin cruce de año.
concepto	—	Obligatorio: servicio, honorarios, arrendamientos, compra, transporte, hoteles.
descripcion, forma_pago	"" / "Contado"	Informativos.
base, iva, total	null	Informe base, o total + IVA, o total con `total_incluye_iva=true`.
tarifa_iva	"0.19"	0.19, 0.05 o 0 (excluido/exento).
total_incluye_iva, iva_excluido	false	Forma de IVA de la factura.
descuentos	"0"	Se restan de la base.
moneda, trm	"COP" / null	USD requiere TRM > 0; conversión simple sin diferencia en cambio.
valor_pagado	null	null = pago total; si es menor, retiene sobre la parte pagada.
retencion_ya_practicada, periodo_cerrado	false	true bloquea (evita doble retención / periodo cerrado).
factura_validada, orden_compra, acta_recibido	true	Estado documental del checklist.
centro_costo	"ADMIN"	Va en cada línea del asiento.

7.5 Salidas

Por cada operación correcta, la consola muestra la explicación, las alertas, los asientos (con su verificación de cuadre), los certificados, el checklist, la autorización de pago y una línea de auditoría. En el directorio `--salida` (por defecto `salidas/`) se guardan:

Archivo	Contenido
`liquidacion_<numero>.json`	Liquidación completa: factura, pago, reglas aplicadas, retenciones, alertas, asientos, certificados, checklist y auditoría (usuario, fecha, UVT, versión de tablas, hash de entrada, entrada y resultado en JSON).
`asientos_<numero>.csv`	Líneas de asiento: tipo, fecha, documento, cuenta, nombre, débito, crédito, tercero, centro de costo, descripción. Separador `;`, UTF-8 con BOM.
`certificados_<numero>.csv`	Solo si hubo retención: tipo, número, año gravable, agente, retenido, concepto, base, tarifa, valor, fundamento, ciudad y fecha.

Con `--solo-mostrar` no se escribe ningún archivo; sirve para revisar antes de guardar.

7.6 Lotes, duplicados y tablas propias

```
# Un lote completo: "operaciones" como lista en el JSON
./motor_liquidador.py calcular --entrada lote.json --salida salidas --usuario "contador@empresa.com"

# Evitar duplicados entre corridas (registra usuario y fecha en el JSON)
./motor_liquidador.py calcular --entrada lote.json --repositorio registradas.json

# Recalcular una factura ya registrada (decisión consciente)
./motor_liquidador.py calcular --entrada factura.json --repositorio registradas.json --autorizar-dupl

# UVT y tarifas propias sin tocar el código
./motor_liquidador.py plantilla-tablas
./motor_liquidador.py calcular --entrada factura.json --tablas tablas_personalizadas.json
```

Si una operación del lote está bloqueada, se imprime su motivo con las alertas y se continúa con las demás: un lote no se cae por una factura mala. El resumen final cuenta liquidaciones correctas y bloqueadas.

7.7 Códigos de salida (para automatizar)

Código	Significado	Qué hacer en un proceso automático
0	Todas las operaciones liquidadas correctamente.	Continuar con el siguiente paso (pagos, contabilización).
1	Error de entrada o archivos: JSON inválido, campo faltante, archivo inexistente.	Corregir la entrada y reintentar; avisar a quien genera el archivo.
2	Una o más operaciones bloqueadas por el motor (revisión humana).	Revisar los mensajes, resolver el caso con el contador y reintentar.

Ejemplo de integración: `./motor_liquidador.py calcular --entrada lote.json --repositorio reg.json --salida salidas >> lote.log 2>&1; echo "Salida: $?"`. Los mensajes operativos van a `stderr` y los resultados a `stdout`, para poder separarlos.

7.8 Ver qué tablas están activas antes de liquidar

Antes de liquidar un lote conviene confirmar qué tablas usará el motor:

```
./motor_liquidador.py tablas                              # tablas por defecto (incluidas en el motor
./motor_liquidador.py tablas --tablas tablas_oficiales.json   # las personalizadas que vas a usar
```

El comando imprime la versión de tablas, la UVT por año, las constantes (IVA, reteIVA y sus mínimos) y cada fila de retefuente y reteICA con tarifa, mínimo, vigencia y fundamento, indicando si vienen "por defecto" o "personalizadas desde" un archivo. Las secciones 6.16 (validación contra la norma) y 6.14 (auditoría) explican cómo usar esa salida como evidencia.

8. Contrato de entradas y salidas para un escenario real

En un caso real, el flujo típico es: el ERP o un formulario produce el JSON de entrada → el script liquida → los JSON/CSV de salida se cargan al sistema contable y la auditoría queda archivada. El contrato completo está en la sección 7; aquí el resumen y el comportamiento ante errores.

8.1 Entrada

Un JSON con `comprador`, `proveedor` y `operaciones` (lista) o `operacion` (única). Los conversores validan cada campo y nombran el que falla. El motor, por su parte, decide qué se puede liquidar y qué no.

8.2 Salida exitosa

- **Consola:** explicación de la liquidación (base, mínimo, tarifa, resultado, fundamento), alertas, asientos con cuadre, certificados, checklist, autorización de pago y línea de auditoría.
- **Archivos:** `liquidacion_*.json`, `asientos_*.csv` y `certificados_*.csv` (este último solo si hay retención).
- **Objeto subyacente:** si se importa el módulo desde Python, `calcular()` devuelve el objeto `Liquidacion` con `resumen()`, `asientos_cuadran` y `puede_pagar` — el mismo contrato del notebook.

8.3 Salida de rechazo

Cuando falta un dato crítico, una regla o el caso está fuera de alcance, la operación se marca como **bloqueada**: se imprime el motivo con sus alertas y el proceso termina con código 2. Los bloqueos típicos: proveedor no residente, factura sin validar, periodo cerrado, duplicado, sin tarifa ICA para el municipio, sin UVT para el año, operación entre grandes contribuyentes. Es una salida correcta: significa que el motor se negó a calcular lo que no puede explicar.

8.4 Qué NO hace

- No firma ni presenta el Formulario 350 ni la declaración municipal: deja datos para el borrador.
- No ejecuta pagos ni contabiliza en un ERP: exporta asientos y certificados en CSV para importar.
- No valida el RUT contra la DIAN: las banderas (SIMPLE, autorretenedor, declarante) se confirman manualmente.
- No maneja notas crédito en `calcular`: para ajustes se usa el motor de notas crédito del proyecto (pendiente de exponer en la CLI en la próxima versión).
- No hay autenticación: `--usuario` es un texto informativo para la auditoría.

9. Los 12 casos de prueba y su lectura

Son los casos de la sección 10 del diseño funcional. Se ejecutan con `./motor_liquidador.py casos`, que los compara contra los valores esperados y sale con código 0 solo si todos coinciden. La columna "Esc." es la fila pedagógica de la tabla de decisión; que el escenario diga "6%" no significa que se haya retenido: el mínimo puede dejar la retención en $0 (caso 3).

#	Caso	Base pago	IVA	Retefuente	ReteIVA	ReteICA	Neto	Esc.	Qué demuestra
1	Servicio declarante	2.000.000	380.000	80.000	0	0	2.300.000	1	Base sin IVA, 4% declarante, SAS normal no retiene IVA.
2	Servicio no declarante	500.000	95.000	30.000	0	0	565.000	2	La condición del proveedor cambia la tarifa: 6%.
3	Bajo mínimo	90.000	17.100	0	0	0	107.100	2	$90.000 < 2 UVT: no hay retención, aunque sea servicio.
4	Exactamente 2 UVT	104.748	19.902,12	4.189,92	2.985,32	0	117.474,88	8	"Inferior a" significa que igual al mínimo sí retiene; redondeo exacto.
5	Honorarios jurídicos	3.000.000	570.000	330.000	0	0	3.240.000	3	Honorarios: 11%, sin mínimo.
6	Arrendamiento vivienda	1.200.000	0	42.000	0	0	1.158.000	5	IVA excluido; 3,5% con mínimo de 10 UVT.
7	Compra declarante	4.000.000	760.000	100.000	0	0	4.660.000	6	Compras: 2,5%, mínimo 10 UVT, base sin IVA.
8	SIMPLE	1.000.000	190.000	0	28.500	0	1.161.500	12	Art. 911: no RF ni ICA; reteIVA sí (responsable de IVA + agente).
9	Autorretenedor	1.000.000	190.000	0	28.500	0	1.161.500	13	No se retiene renta a un autorretenedor; reteIVA puede aplicar.
10	Total con IVA	1.000.000	190.000	40.000	0	0	1.150.000	1	Total $1.190.000 con IVA: base = 1.190.000 / 1,19.

11	Pago parcial	500.000	95.000	20.000	14.250	0	560.750	8	Se retiene sobre el 50% pagado, no sobre la factura completa.
12	ReteICA Bogotá	2.000.000	380.000	80.000	0	19.320	2.280.680	10	ICA municipal 0,966% (pedagógico) sobre la base sin IVA.

10. Criterios de aceptación

Los 10 criterios de la sección 11 del diseño quedan verificados en el proyecto (el comando `casos` cubre los casos base y las variantes están probadas en el notebook y en la batería de pruebas del script).

#	Criterio	Cómo se verifica
1	Servicio $2.000.000 + IVA 19%, proveedor declarante, comprador SAS normal → IVA $380.000, RF $80.000, neto $2.300.000.	Caso 1 de `casos`.
2	El mismo caso con comprador gran contribuyente → reteIVA $57.000 y neto $2.243.000.	Variante con `agente_reteiva=true, gran_contribuyente=true`.
3	Servicio de $90.000 → no se practica retefuente ni reteIVA (aunque el comprador sea agente).	Caso 3 y variante con gran contribuyente.
4	Proveedor SIMPLE → no se practica retefuente ni reteICA; reteIVA $28.500 si aplica.	Caso 8 y variante con agente de reteICA.
5	Proveedor autorretenedor → no se practica retefuente.	Caso 9.
6	Los asientos de causación y pago siempre cuadran.	El motor lanza excepción si no cuadran; los 12 casos los imprimen cuadrados.
7	El certificado de renta incluye los siete requisitos del Art. 381.	`requisitos_art_381()` devuelve 7 datos no vacíos.
8	El certificado de reteIVA discrimina base sin IVA, IVA, tarifa y valor.	Caso 4: base $104.748, IVA $19.902,12, tarifa 15%, valor $2.985,32.
9	Se guarda la versión de la tabla y la UVT usada.	Auditoría de cada liquidación y del JSON de salida.
10	Se bloquea el cálculo si falta una regla o un dato crítico.	Bloqueos verificados: no residente, factura sin validar, periodo cerrado, sin tarifa ICA, sin UVT, duplicado (código de salida 2).

11. Limitaciones y camino a producción

La v1 implementa el alcance del diseño funcional: compras nacionales, proveedores residentes, comprador persona jurídica con contabilidad, IVA general, retefuente, reteIVA y reteICA (según calidad), proveedores declarantes / no declarantes / SIMPLE / autorretenedores, facturas y notas crédito simples, asientos, certificados y trazabilidad. Todo lo demás **bloquea** y pide revisión humana.

No cubre (por diseño): pagos al exterior y convenios, régimen SIMPLE a fondo, autorretención especial, hidrocarburos, construcción, entidades públicas y salud especial, servicios temporales/aseo/vigilancia con AIU, operaciones entre grandes contribuyentes con reglas especiales, prorrata de IVA, moneda extranjera con diferencia en cambio, declaraciones automáticas y planeación tributaria.

Limitaciones concretas de esta implementación didáctica:

- La tabla de ICA solo trae la tarifa pedagógica de Bogotá para servicios (mínimo sin configurar); otras actividades o municipios bloquean hasta cargarlas con `--tablas`.
- La tabla de retefuente cubre los conceptos del diseño y algunos de la tabla de referencia, no las 77 filas de la tabla oficial.
- La excepción de honorarios (3.300 UVT, Art. 383 ET) está documentada pero no aplicada.
- La exclusión de reteIVA entre entidades estatales, grandes contribuyentes y emisoras de tarjetas solo está implementada para el caso de ambos grandes contribuyentes.
- No hay persistencia en base de datos ni autenticación: la persistencia es por archivos (JSON/CSV y repositorio de duplicados).
- Las notas crédito aún no están expuestas como subcomando; el motor de ajuste existe en el proyecto.

Camino a producción (fases 0–4 de la hoja de ruta del diseño):

1. Validar 20 facturas reales a mano y firmar el alcance con un contador.
2. Cargar las tablas oficiales completas con vigencias y fundamentos vía `--tablas`.
3. Validar RUT y calidad de agente contra fuentes oficiales (hoy son banderas manuales).
4. Persistir liquidaciones y auditoría en una base de datos y agregar roles y bloqueo de periodos.
5. Generar PDF/XML de certificados y conectar los CSV con el ERP.
6. Marcha blanca: correr un mes en paralelo con el proceso manual y comparar.
7. Definir responsables de tablas y actualizar UVT y tarifas con cada cambio normativo.

12. Glosario

Término	Significado en este proyecto
UVT	Unidad de valor tributario. Los mínimos de retención se expresan en UVT. UVT 2026 = $52.374.
Retefuente (RF)	Retención en la fuente a título del impuesto sobre la renta, practicada por el agente al pagar.
ReteIVA (RIVA)	Retención del IVA (15% del IVA facturado) que practica el comprador cuando es agente de reteIVA.
ReteICA (RICA)	Retención del impuesto de industria y comercio, de nivel municipal, según la actividad del vendedor.
Agente de retención	Quien la norma obliga a practicar la retención y consignarla (cada retención tiene sus propios agentes).
Declarante / no declarante	Condición del proveedor frente al impuesto de renta; define la columna de tarifa (4% vs 6%, 2,5% vs 3,5%, 11% vs 10%).
SIMPLE	Régimen Simple de Tributación. No se le practica retefuente ni reteICA (Art. 911 ET); reteIVA según el caso.
Autorretenedor	Autorizado a retenerse a sí mismo. Código 15 = general; código 59 = especial (no exime de todo).
Gran contribuyente	Calidad asignada por la DIAN; suele ser agente de reteIVA y tiene reglas de exclusión entre grandes.
Base gravable	Valor sobre el que se aplica la tarifa. Para retenciones: antes de IVA, menos descuentos y devoluciones.
Mínimo	Valor (en UVT) por debajo del cual no hay retención. "Inferior a" → si es igual al mínimo, sí retiene.
IVA excluido / exento	Operaciones sin IVA (tarifa 0%). El arrendamiento de vivienda es excluido.
Total con IVA incluido	Factura cuyo total ya contiene el IVA; la base se extrae dividiendo por (1 + tarifa).
Neto	Valor realmente pagado: total pagado menos las tres retenciones.
Causación	Asiento que reconoce el gasto/costo y el pasivo con el proveedor al momento de la obligación.
Pasivo de retención	Cuenta 236x: la retención no es un gasto; es una deuda con la administración mientras se declara y paga.
TRM	Tasa representativa del mercado, usada para convertir USD a COP en la v1 (sin diferencia en cambio).
RUT	Registro Único Tributario del proveedor; evidencia su condición (códigos 15, 47, 59…).
CIIU	Clasificación industrial internacional uniforme; la actividad del vendedor define la tarifa de ICA.
Formulario 350	Declaración mensual de retenciones (renta e IVA) ante la DIAN. El script solo deja los datos del borrador.
CUFE	Código único de validación de la factura electrónica; se exige "factura validada" como entrada documental.
Auditoría	Registro de quién, cuándo, con qué regla, qué UVT y qué datos se obtuvo cada resultado.
JSON	Formato de datos de la entrada y de la salida de auditoría; legible por humanos y máquinas.
CSV	Archivo de texto tabular (separador ;) para abrir en Excel e importar al ERP.
Código de salida	Número que el script devuelve al sistema operativo (0 / 1 / 2) para automatizar decisiones.
stdout / stderr	Canales de salida: resultados a stdout, mensajes operativos y errores a stderr.

13. Referencias del proyecto y normativas

Archivos del repositorio

Archivo	Qué es
`motor_liquidador.py`	El script estudiado en este documento (motor + CLI + persistencia).
`liquidador.ipynb`	El mismo motor como notebook didáctico, con los resultados guardados.
`docs/diseno-funcional.md`	Diseño funcional: entradas, reglas R1–R20, validaciones, casos y criterios de aceptación.
`docs/tabla-retenciones-2026.md`	Tabla de referencia de conceptos, tarifas y mínimos con la línea de tiempo del Decreto 572.
`docs/matriz-comprador-vendedor.md`	Matriz comprador–vendedor y lectura crítica de la tabla.
`docs/casos-practicos.md`	16 casos resueltos a mano con asientos completos.
`Control_Retenciones_16_Casos.xlsx` / `casos.csv`	Tabla de control de los 16 casos y datos para importación.
`README.md`	Resumen del proyecto, reglas base y estado.

Referencias normativas citadas por el proyecto

- Estatuto Tributario: arts. 381 (certificado de renta), 383 (excepción de honorarios), 387 (arrendamiento de vivienda), 392 (tarifas de retención), 401 (compras), 437-1 y 437-2 (reteIVA), 462-1 (IVA) y 911 (SIMPLE).
- Decreto 572 de 2025 (mínimo de servicios de 2 UVT y su línea de tiempo de suspensión/restablecimiento durante 2026).
- Decreto 1625 de 2016, art. 1.6.1.12.13 (certificado de reteIVA, según la documentación del proyecto).
- Auto 30229 de 2026 y UVT 2026 = $52.374.
- Acuerdo 724 de 2026 (Bogotá): tarifa pedagógica de ICA 0,966% usada en los ejemplos.

> **Recordatorio final.** Los fundamentos legales aquí citados provienen de la documentación educativa del repositorio y deben verificarse contra la norma vigente antes de cualquier uso real. La tabla oficial de retenciones y la norma actualizada prevalecen sobre este material. El liquidador es una herramienta de consistencia: la decisión final siempre es del contador.